PROJECT DOCUMENTATION REPORT

AI Research Paper Skeptic Agent

A multi-agent RAG system that reads a research paper and produces a structured, evidence-checked skeptical review

Track	AAI
Domain	Academic research support
Difficulty Level	Basic + Intermediate
Recommended Duration	10 Days
Mode	Individual or Group of 3 Students
Repository	**Scholar_Research_Agent**

1. Problem Statement

Students and early-stage researchers often summarize academic papers uncritically, restating claims without checking whether the paper's own evidence actually supports them. The AI Research Paper Skeptic Agent addresses this by reading a paper and generating a skeptical, evidence-grounded review that surfaces the main claim, method, supporting evidence, limitations, and open questions a careful reader should ask.

The system is built as a multi-agent pipeline rather than a single summarization call, so that claim extraction, evidence retrieval, and grounding verification are handled by separate, auditable stages instead of one opaque generation step.

1.1 Real-World Impact

The agent keeps a human in the loop for judgment calls while automating the repetitive, error-prone work of cross-checking a paper's claims against its own text. This is directly useful for students doing literature reviews, journal clubs, and early-career researchers screening papers before deeper reading — the tool flags where a claim is well-supported, weakly supported, or unsupported by retrieved evidence, rather than making a final accept/reject decision itself.

2. Dataset / Reference Source

Source: arXiv API and PDF documents (https://info.arxiv.org/help/api/index.html).

How it is used: papers are fetched by arXiv ID/URL or uploaded directly; text is extracted, chunked, embedded, and stored in a vector index so that claims can be mapped back to specific passages as retrieved evidence.

Synthetic fallback: generate_sample_papers.py creates synthetic sample paper text files in data/sample_papers/ for testing and demos when live arXiv access is unavailable. These are clearly synthetic starter data, not real publications.

Output target: a structured review object (claim, method, evidence, limitations, questions) plus a groundedness flag per claim, persisted to SQLite for logging and feedback.

3. Recommended Tools and Technologies

Python, an LLM API (OpenAI-compatible), the AutoGen multi-agent framework, prompt templates, Pydantic schemas for structured JSON outputs, a vector store (FAISS or Chroma), SQLite for memory/logging, Streamlit for the UI, and standard logging. An optional MCP (Model Context Protocol) layer exposes the tool functions to agents over a client/server boundary; this and web search are stretch goals and are disabled by default.

4. System / Pipeline Architecture

A user submits a paper (or arXiv link) to the Orchestrator, which coordinates a four-agent AutoGen GroupChat — Retriever, Skeptic, Guardrail, and Reporter. The agents share a RAG vector store for evidence lookup and a SQLite-backed memory store for session state, logs, and feedback. Every claim the Skeptic agent proposes is passed through a guardrail check before it reaches the final structured review.

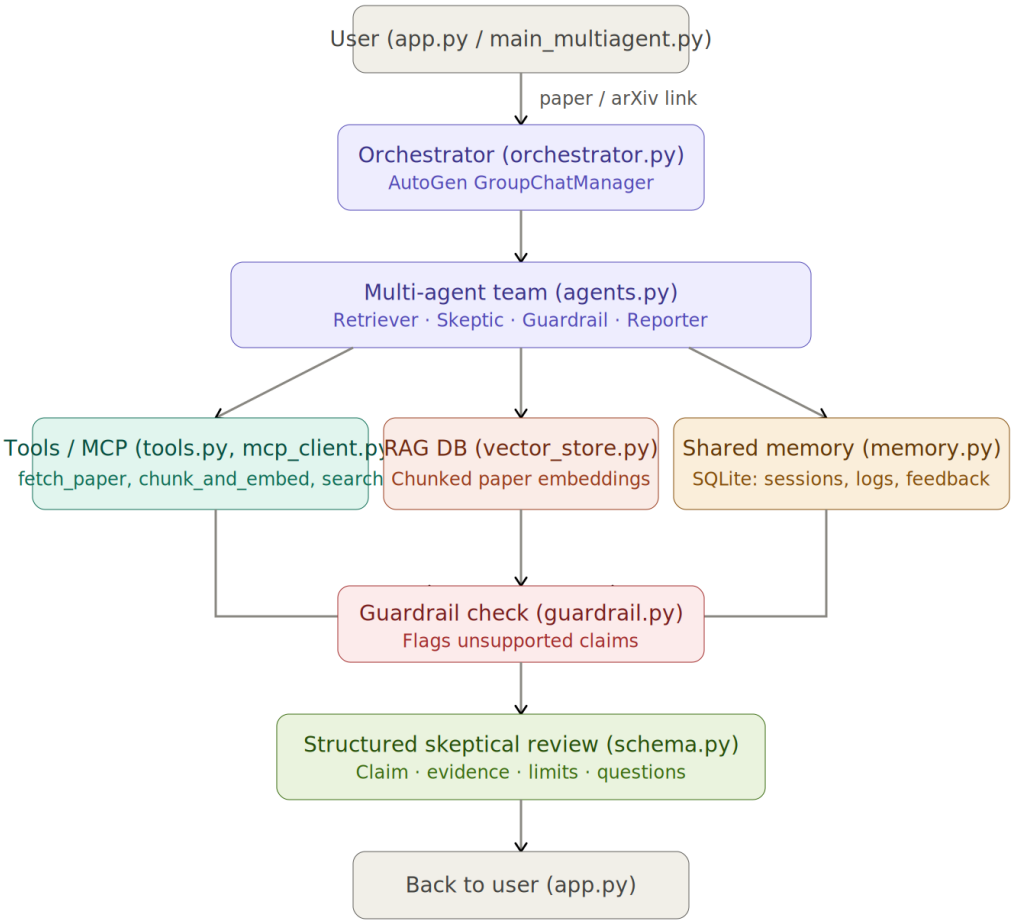

Figure 1 — End-to-end system architecture: user → orchestrator → multi-agent team → vector store / memory → guardrail check → structured review.

4.1 High-Level Project Flow

- User provides a goal/input (a paper or arXiv link).
- The agent understands the task and asks clarifying questions if information is missing.
- Tools, retrieval, and memory are used to gather evidence for each claim.
- Guardrails check that every claim is adequately grounded in retrieved evidence.
- A structured output (claim, method, evidence, limitations, questions) is produced.
- The result and any user feedback are logged for later review.

4.2 Core Modules

- User input form (Streamlit UI / CLI)
- Prompt workflow for each agent role
- Tool / function layer (ingestion, chunking, retrieval, optional web search)
- Memory / retrieval (vector store + SQLite)
- Guardrails and fallback logic
- Logs / evaluation sheet

5. AI / ML / Agent Component

The AI/agent logic is concentrated in four places, each chosen because it maps to a distinct failure mode of naive paper summarization:

PDF RAG (ingestion.py, chunking.py, vector_store.py, retriever.py): the paper is chunked (by section where possible, else sliding windows), embedded, and indexed so that any claim can be checked against the paper's actual text rather than the model's memorized or hallucinated understanding of it.

Multi-agent claim–evidence mapping (agents.py, orchestrator.py): a Retriever agent gathers passages, a Skeptic agent drafts claims and limitations, a Guardrail agent verifies grounding, and a Reporter agent assembles the final structured review — separating claim generation from claim verification.

Limitation extraction: the Skeptic agent is prompted specifically to surface method weaknesses, missing baselines, small sample sizes, and unaddressed confounders, rather than only summarizing stated conclusions.

Unsupported-claim guardrail (guardrail.py): check_grounding(claim, chunk_ids) rejects or flags any claim whose cosine similarity to its cited evidence falls below GROUNDING_THRESHOLD (default 0.6), giving a quantitative, adjustable check rather than a purely subjective one.

6. Repository Structure

```
Scholar_Research_Agent/
│
├── data/
│   ├── sample_papers/          # synthetic sample papers
│   └── memory.db               # SQLite file, created at first run
│
├── src/
│   ├── agents.py
│   ├── baseline_agent.py
│   ├── chunking.py
│   ├── config.py
│   ├── guardrail.py
│   ├── ingestion.py
│   ├── logger.py
│   ├── mcp_client.py
│   ├── memory.py
│   ├── orchestrator.py
│   ├── retriever.py
│   ├── schema.py
│   ├── tools.py
│   └── vector_store.py
│
├── app.py                      # Streamlit UI
├── current_reqs.txt
├── generate_sample_papers.py
├── main_multiagent.py          # CLI entry point
├── run_ingestion.py
├── test_guardrail.py
├── valid_queries.md
├── verify_baseline.py
└── verify_db.py
```

6.1 File Dependency Map

config.py is read by every layer (dashed connections in the diagram below); memory.py feeds session state and logs back up to the entry points (app.py and main_multiagent.py).

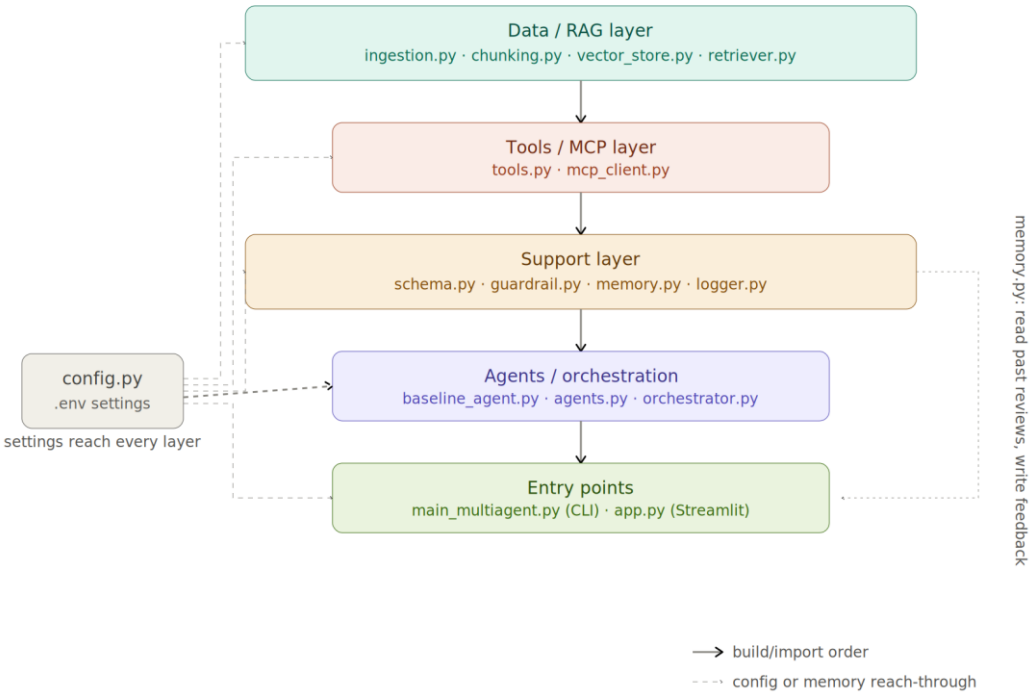

Figure 2 — File dependency graph across the six build layers: configuration, data/RAG pipeline, tools/MCP, support (schema, guardrail, memory, logging), agents/orchestration, and entry points.

6.2 File-by-File Summary

Layer 0 — Configuration

File	Purpose	Key configuration
config.py	Single source of truth for all settings, loaded from .env; fails fast if LLM_API_KEY is missing.	LLM_MODEL_NAME, LLM_API_KEY, EMBEDDING_MODEL, CHUNK_SIZE/OVERLAP, VECTOR_BACKEND, RETRIEVAL_K, GROUNDING_THRESHOLD, SQLITE_DB_PATH, USE_MCP, MAX_TURNS

Layer 1 — Data / RAG Pipeline

File	Purpose	Depends on
ingestion.py	fetch_paper() pulls a PDF from arXiv ID/URL or upload; extracts text + metadata.	config.py
chunking.py	chunk_and_embed() splits text by section/sliding window and embeds each chunk.	config.py
vector_store.py	Wraps FAISS/Chroma — add_chunks(), similarity_search().	chunking.py, config.py
retriever.py	retrieve_passages(query, paper_id, k) — semantic search for claim-evidence mapping.	vector_store.py

Layer 2 — Tools / MCP

File	Purpose	Depends on
tools.py	Wraps ingestion, chunking, and retrieval as typed callable functions for agent registration.	ingestion.py, chunking.py, retriever.py
mcp_client.py	Connects to an MCP server as a client and exposes tool functions to agents.	mcp library

Layer 3 — Support

File	Purpose	Depends on
schema.py	Pydantic models for Claim, Review, and tool I/O shapes.	none internal
guardrail.py	check_grounding(claim, chunk_ids) rejects claims lacking retrieved support.	retriever.py, schema.py
memory.py	SQLite schema + read/write for papers, reviews, feedback, agent_logs.	schema.py, config.py
logger.py	Standardized JSON and console logging utility.	none internal

Layer 4 — Agents and Orchestration

File	Purpose	Depends on
baseline_agent.py	Single-agent baseline ResearchAssistant using RAG (Ollama), for comparison.	config.py, retriever.py

File	Purpose	Depends on
agents.py	Defines the AutoGen Retriever, Skeptic, Guardrail, and Reporter agents.	tools.py/mcp_client.py, guardrail.py, schema.py, memory.py
orchestrator.py	Builds the AutoGen GroupChat + GroupChatManager driving paper → Review.	agents.py, memory.py, logger.py

Layer 5 — Entry Points & Utilities

File	Purpose
app.py	Streamlit UI — upload/paste a paper, view the review, groundedness flags, collect feedback.
main_multiagent.py	CLI entry point for the multi-agent workflow.
run_ingestion.py	Runs the ingestion pipeline on all sample papers.
generate_sample_papers.py	Creates synthetic sample papers in data/sample_papers/.
test_guardrail.py	Tests grounding-check logic against mock text.
verify_baseline.py	Evaluates the baseline single-agent workflow.
verify_db.py	Verifies end-to-end vector database ingestion and retrieval.

7. Configuration Reference (.env.example)

All configuration lives in one file and is loaded once by config.py. The MCP and web-search integrations are optional stretch goals and are disabled by default so the base system runs without extra credentials.

```
LLM_MODEL_NAME=gpt-4o-mini
LLM_API_KEY=your_key_here

EMBEDDING_MODEL=text-embedding-3-small
CHUNK_SIZE=500
CHUNK_OVERLAP=50

VECTOR_BACKEND=faiss
VECTOR_DB_PATH=./data/vector_store

RETRIEVAL_K=5
GROUNDING_THRESHOLD=0.6

SQLITE_DB_PATH=./data/memory.db

USE_MCP=false
MCP_HOST=127.0.0.1
MCP_PORT=8765

SEARCH_API_KEY=

MAX_TURNS=12
MAX_RETRY_ATTEMPTS=2
LOG_LEVEL=INFO
```

8. How to Run the Project

- Copy .env.example to .env and set LLM_API_KEY (leave USE_MCP=false and SEARCH_API_KEY blank for the basic build).
- Install dependencies from current_reqs.txt.
- Run generate_sample_papers.py to populate data/sample_papers/ with synthetic test data (optional if using real arXiv papers).
- Run run_ingestion.py to build the vector store from the sample/reference papers.
- Launch the CLI with main_multiagent.py, or the UI with streamlit run app.py.
- Run verify_db.py, verify_baseline.py, and test_guardrail.py to validate each pipeline stage independently.

9. Validation and Evaluation Method

Validation is split across the three verification scripts rather than a single end-to-end check, so each stage of the pipeline can be debugged independently:

verify_db.py: confirms that ingestion, chunking, embedding, and retrieval return the expected passages for known sample queries.

verify_baseline.py: runs the single-agent baseline for comparison against the multi-agent pipeline's output quality.

test_guardrail.py: feeds mock claims with known-good and known-bad evidence to check_grounding() and confirms the threshold correctly flags unsupported claims.

A recommended additional scenario check: run the same sample paper through both the baseline agent and the full multi-agent pipeline, and compare whether the Skeptic + Guardrail flow catches unsupported claims that the baseline silently accepts.

10. Limitations and Responsible Use

- The groundedness check is a cosine-similarity threshold, not a logical entailment check — it can be fooled by passages that are topically similar but do not actually support the claim.
- Retrieval quality depends on chunking strategy; poorly segmented PDFs (multi-column layouts, scanned images without OCR) will degrade both retrieval and grounding accuracy.
- The agent is a screening aid, not a substitute for peer review or domain-expert judgment — flagged claims still require human verification before being acted on.
- LLM-generated limitations and questions may still miss subtle methodological issues that require deep subject-matter expertise.
- The system should not be used as the sole basis for high-stakes decisions (e.g., clinical, legal, or policy conclusions) without expert review.

11. Future Improvements

- Enable the MCP tool layer and web_search tool for cross-referencing claims against related literature, not just the paper itself.
- Add a natural-language-inference model as a second grounding check alongside cosine similarity, to catch topically-similar-but-non-supporting evidence.
- Add a feedback loop where user corrections to flagged claims are used to recalibrate GROUNDING_THRESHOLD per domain.

- Extend baseline_agent.py comparisons into a small benchmark set with human-labeled ground truth for claim groundedness.

12. Team Members

[Add team member names and roles here.]

Prepared from: Project_07_CP_AAI_AI_Research_Paper_Skeptic_Agent.md, Skeptic_Agent_FileStructure.md, and the accompanying architecture and file-dependency diagrams.